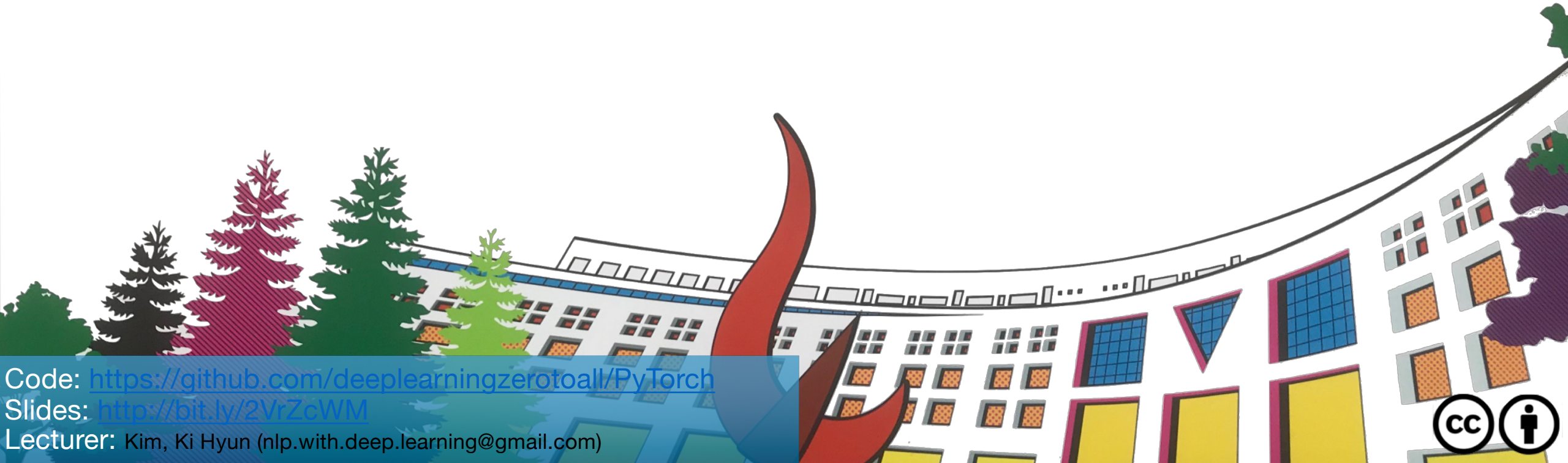
ML/DL for Everyone Season2

with PYTORCH

Softmax Classification



Code: https://github.com/deeplearningzerotoall/PyTorch
Slides: http://bit.ly/2VrZcWM
Lecturer: Kim, Ki Hyun (nlp.with.deep.learning@gmail.com)

Softmax Classification

- Softmax
- Cross Entropy
- Low-level Implementation
- High-level Implementation
- Training Example

Import

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
```

```
# For reproducibility
torch.manual_seed(1)
```

```
<torch._C.Generator at 0x7f8740076fb0>
```

Discrete Probability Distribution

이산적인 확률 분포
>> 딱 떨어진 숫자들만 활용 가능

① Binary Classification — Sigmoid

클래스가 0과 1, 두 개뿐이라면 **클래스 1의 확률 하나만 계산하면 돼**.

예를 들어,

$$P(y=1) = 0.8$$

그렇다면 클래스 0의 확률은 자동으로 결정되지.

$$P(y=0) = 1 - 0.8 = 0.2$$

따라서 Sigmoid는 출력값 하나만 필요해.

② Multiclass Classification — Softmax

그런데 클래스가 0, 1, 2라면?

클래스 2의 확률이 0.7이라는 것만으로는 나머지 클래스의 확률을 알 수 없어.

따라서 **클래스마다 각각의 점수를 계산한 뒤 Softmax로 확률을 변환**하는 거야.

Softmax

*max 값을 뽑아주는데 soft하게 뽑아줌
소프트맥스를 거치면 이 점수들이 [0.65, 0.24, 0.11]처럼 합이
정확히 1(100%)이 되는 확률 분포로 바뀝니다.

Convert numbers to probabilities with softmax.

$$P(class = i) = \frac{e^i}{\sum e^i}$$

```
z = torch.FloatTensor([1, 2, 3])
```

PyTorch has a `softmax` function.

```
hypothesis = F.softmax(z, dim=0)
print(hypothesis)
```

```
tensor([0.0900, 0.2447, 0.6652])
```

Since they are probabilities, they should add up to 1. Let's do a sanity check.

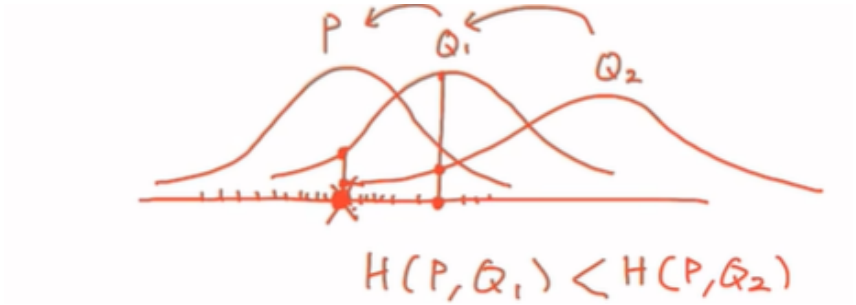
```
hypothesis.sum()
```

```
tensor(1.)
```

Cross Entropy

$$H(P,Q) = -\mathbb{E}_{\mathrm{x}\sim P(\mathrm{x})}[\log Q(\mathrm{x})] = -\sum_{x\in\mathcal{X}} P(x)\log Q(x)$$

모델이 예측한 확률 분포가 "진짜 정답(실제값)과 얼마나 다른지"를 측정



Cross Entropy Loss (Low-level)

For multi-class classification, we use the cross entropy loss.

$$L = \frac{1}{N} \sum -y \log(\hat{y})$$

where $\hat{y}$ is the predicted probability and y is the correct probability (0 or 1).

```
z = torch.rand(3, 5, requires_grad=True)
hypothesis = F.softmax(z, dim=1)
print(hypothesis)
```

```
tensor([[0.2645, 0.1639, 0.1855, 0.2585, 0.1277],
        [0.2430, 0.1624, 0.2322, 0.1930, 0.1694],
        [0.2226, 0.1986, 0.2326, 0.1594, 0.1868]], grad_fn=<SoftmaxBackwa
rd>)
```

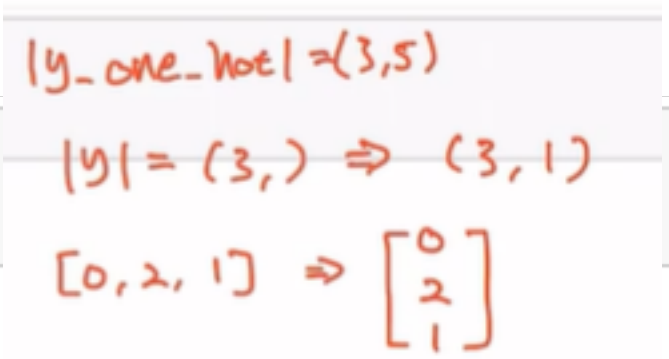
*prediction dim=1의 방향

```
y = torch.randint(5, (3,)).long()
print(y)
```

```
tensor([0, 2, 1])
```

Cross Entropy Loss (Low-level)

```
y_one_hot = torch.zeros_like(hypothesis)
y_one_hot.scatter_(1, y.unsqueeze(1), 1)
```




```
tensor([[1., 0., 0., 0., 0.],
        [0., 0., 1., 0., 0.],
        [0., 1., 0., 0., 0.]])
```

```
cost = (y_one_hot * -torch.log(hypothesis)).sum(dim=1).mean()
print(cost)
```

```
tensor(1.4689, grad_fn=<MeanBackward1>)
```

Cross-entropy Loss with torch.nn.functional

```
# Low level
torch.log(F.softmax(z, dim=1))
```

```
tensor([[-1.3301, -1.8084, -1.6846, -1.3530, -2.0584],
        [-1.4147, -1.8174, -1.4602, -1.6450, -1.7758],
        [-1.5025, -1.6165, -1.4586, -1.8360, -1.6776]], grad_fn=<LogBackw
ard>)
```

```
# High level
F.log_softmax(z, dim=1)
```

```
tensor([[-1.3301, -1.8084, -1.6846, -1.3530, -2.0584],
        [-1.4147, -1.8174, -1.4602, -1.6450, -1.7758],
        [-1.5025, -1.6165, -1.4586, -1.8360, -1.6776]],
       grad_fn=<LogSoftmaxBackward>)
```

Cross-entropy Loss with torch.nn.functional

```
# Low level
(y_one_hot * -torch.log(F.softmax(z, dim=1))).sum(dim=1).mean()
```

```
tensor(1.4689, grad_fn=<MeanBackward1>)
```

```
# High level
F.nll_loss(F.log_softmax(z, dim=1), y)
```

```
tensor(1.4689, grad_fn=<NllLossBackward>)
```

PyTorch also has `F.cross_entropy` that combines `F.log_softmax()` and `F.nll_loss()`.

```
F.cross_entropy(z, y)
```

```
tensor(1.4689, grad_fn=<NllLossBackward>)
```

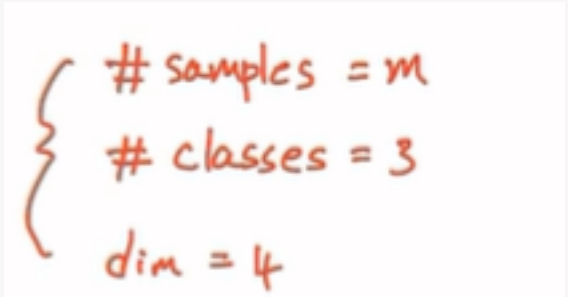
Cross-entropy Loss with torch.nn.functional

```
F.cross_entropy(z, y)
```

```
tensor(1.4689, grad_fn=<NllLossBackward>)
```

Training with Low-level Cross Entropy Loss

```
x_train = [[1, 2, 1, 1],
           [2, 1, 3, 2],
           [3, 1, 3, 4],
           [4, 1, 5, 5],
           [1, 7, 5, 5],
           [1, 2, 5, 6],
           [1, 6, 6, 6],
           [1, 7, 7, 7]]
y_train = [2, 2, 2, 1, 1, 1, 0, 0]
x_train = torch.FloatTensor(x_train) *정수
y_train = torch.LongTensor(y_train)
```



Training with Low-level Cross Entropy Loss

```python
# 모델 초기화
W = torch.zeros((4, 3), requires_grad=True)
b = torch.zeros(1, requires_grad=True)
# optimizer 설정
optimizer = optim.SGD([W, b], lr=0.1)

nb_epochs = 1000
for epoch in range(nb_epochs + 1):

    # Cost 계산 (1)
    hypothesis = F.softmax(x_train.matmul(W) + b, dim=1) # or .mm or @
    y_one_hot = torch.zeros_like(hypothesis)
    y_one_hot.scatter_(1, y_train.unsqueeze(1), 1)
    cost = (y_one_hot * -torch.log(F.softmax(hypothesis, dim=1))).sum(dim=1).mean()

    # cost로 H(x) 개선
    optimizer.zero_grad()
    cost.backward()
    optimizer.step()

    # 100번마다 로그 출력
    if epoch % 100 == 0:
        print('Epoch {:4d}/{} Cost: {:.6f}'.format(
            epoch, nb_epochs, cost.item()
        ))
```

특징의 갯수, 출력 갯수

*행렬곱이 아니라 같은 원소의 위치에 곱할 것

*즉, 1과 곱한 위치의 손실만 남게 됨 (8,3) 유지

*sum하면 가로로 3개씩 더함

```
Epoch    0/1000 Cost: 1.098612
Epoch  100/1000 Cost: 0.901535
Epoch  200/1000 Cost: 0.839114
Epoch  300/1000 Cost: 0.807826
Epoch  400/1000 Cost: 0.788472
Epoch  500/1000 Cost: 0.774822
Epoch  600/1000 Cost: 0.764449
Epoch  700/1000 Cost: 0.756191
Epoch  800/1000 Cost: 0.749398
Epoch  900/1000 Cost: 0.743671
Epoch 1000/1000 Cost: 0.738749
```

```python
# 1. 클래스별 예측 확률 (8, 3)
hypothesis = F.softmax(x_train.matmul(W) + b, dim=1)

# 2. 정답과 같은 크기의 0 행렬 생성 (8, 3)
y_one_hot = torch.zeros_like(hypothesis)

# 3. 정답 클래스 위치에 1 삽입
y_one_hot.scatter_(1, y_train.unsqueeze(1), 1)

# 4. Cross Entropy Loss 계산
cost = (y_one_hot * -torch.log(hypothesis)).sum(dim=1).mean()
```

① hypothesis — Softmax 예측 확률 (8, 3)

데이터	Class 0	Class 1	Class 2	정답
1	0.10	0.20	**0.70**	2
2	0.20	0.20	**0.60**	2
3	0.10	0.10	**0.80**	2
4	0.20	**0.50**	0.30	1
5	0.40	**0.40**	0.20	1
6	0.05	**0.90**	0.05	1
7	**0.60**	0.20	0.20	0
8	**0.80**	0.10	0.10	0

```
[
 [0, 0, 1],
 [0, 0, 1],
 [0, 0, 1],
 [0, 1, 0],
 [0, 1, 0],
 [0, 1, 0],
 [1, 0, 0],
 [1, 0, 0]
]
```

정답 클래스 예측 확률	손실 $-\log(p)$	의미
0.1 (10%)	2.303	정답에 낮은 확률 부여
0.7 (70%)	0.357	정답이지만 확신이 부족
0.9 (90%)	0.105	정답에 높은 확률 부여
1.0 (100%)	0	정답에 완전한 확률 부여

Training with F.cross_entropy

```python
# 모델 초기화
W = torch.zeros((4, 3), requires_grad=True)
b = torch.zeros(1, requires_grad=True)
# optimizer 설정
optimizer = optim.SGD([W, b], lr=0.1)

nb_epochs = 1000
for epoch in range(nb_epochs + 1):

    # Cost 계산 (2)
    z = x_train.matmul(W) + b # or .mm or @
    cost = F.cross_entropy(z, y_train)

    # cost로 H(x) 개선
    optimizer.zero_grad()
    cost.backward()
    optimizer.step()

    # 100번마다 로그 출력
    if epoch % 100 == 0:
        print('Epoch {:4d}/{} Cost: {:.6f}'.format(
            epoch, nb_epochs, cost.item()
        ))
```

*One Hot Vector를 만드는 과정 생략 가능

```
Epoch    0/1000 Cost: 1.098612
Epoch  100/1000 Cost: 0.761050
Epoch  200/1000 Cost: 0.689991
Epoch  300/1000 Cost: 0.643229
Epoch  400/1000 Cost: 0.604117
Epoch  500/1000 Cost: 0.568255
Epoch  600/1000 Cost: 0.533922
Epoch  700/1000 Cost: 0.500291
Epoch  800/1000 Cost: 0.466908
Epoch  900/1000 Cost: 0.433507
Epoch 1000/1000 Cost: 0.399962
```

High-level Implementation with nn.Module

```
class SoftmaxClassifierModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(4, 3) # Output0| 3!

    def forward(self, x):
        return self.linear(x)
```

```
model = SoftmaxClassifierModel()
```

High-le dule

```
# optimizer 설정
optimizer = optim.SGI

nb_epochs = 1000
for epoch in range(nb

    # H(x) 계산
    prediction = mode

    # cost 계산
    cost = F.cross_er

    # cost로 H(x) 개선
    optimizer.zero_gr
    cost.backward()
    optimizer.step()

    # 20번마다 로그 출력
    if epoch % 100 ==
        print('Epoch
            epoch, nb
        ))
```

```
Epoch    0/1000 Cost:
Epoch  100/1000 Cost:
Epoch  200/1000 Cost:
Epoch  300/1000 Cost:
Epoch  400/1000 Cost:
Epoch  500/1000 Cost:
Epoch  600/1000 Cost:
Epoch  700/1000 Cost:
Epoch  800/1000 Cost:
Epoch  900/1000 Cost: 0.269521
Epoch 1000/1000 Cost: 0.241922
```

① Binary Classification — Sigmoid

클래스가 0과 1, 두 개뿐이라면 **클래스 1의 확률 하나만 계산하면 돼**.

예를 들어,

$$P(y=1)=0.8$$

그렇다면 클래스 0의 확률은 자동으로 결정되지.

$$P(y=0)=1-0.8=0.2$$

따라서 Sigmoid는 출력값 하나만 필요해.

② Multiclass Classification — Softmax

그런데 클래스가 0, 1, 2라면?

클래스 2의 확률이 0.7이라는 것만으로는 나머지 클래스의 확률을 알 수 없어.

따라서 **클래스마다 각각의 점수를 계산한 뒤 Softmax로 확률을 변환**하는 거야.

	Multiclass Classification
	0, 1, 2
	`Softmax`
`opy`	`Cross Entropy`
	(8, 3)
	0, 1, 2